



TypeScript

O que é?

- ▶ Linguagem de programação **fortemente tipada**;
- ▶ Adiciona sintaxe sobre o **JavaScript** permitindo o uso de tipos;
- ▶ Permite especificar os tipos dos dados que estão sendo passados ao longo do código;
- ▶ Reporta erros quando os tipos não batem;
- ▶ Possui **checagem de tipo em tempo de compilação**.

Como usar?

- ▶ Comumente, utiliza-se o compilador TypeScript oficial, que **transpila código TypeScript em JavaScript**;

- ▶ Pode ser instalado através do npm:

```
npm install typescript --save-dev
```

- ▶ **npx tsc** – exibe a versão do compilador.

Configurando o Compilador

- ▶ A configuração é feita no arquivo [tsconfig.json](#);
- ▶ O arquivo acima pode ser criado através do comando:

npx tsc --init

- ▶ Exemplo:

```
{
  "include": ["src"],
  "compilerOptions": {
    "outDir": "./build"
  }
}
```

Tipos simples (primitivos)

- ▶ 3 tipos principais:
 - ▶ **boolean**: true ou false;
 - ▶ **number**: números inteiros e decimais;
 - ▶ **string**: textos.

Formas de atribuição

► Explícita:

```
let firstName: string = "Dylan";
```

► Implícita:

```
let firstName = "Dylan";
```

Erro na atribuição de tipo

- ▶ No exemplo abaixo será disparado um erro pelos tipos não coincidirem:

```
let nome: string = "Arnaldo";  
nome = 31;
```

- ▶ No JavaScript, isto não gera mensagem de erro.

Tipos especiais

- ▶ **any**: desabilita a verificação de tipos;
- ▶ **unknown**: para tipos de dados desconhecidos;
- ▶ **never**: lança um erro se a variável nunca foi atribuída;
- ▶ **undefined**: equivalente ao mesmo tipo no JavaScript.

Arrays

- Sintaxe específica para definir arrays:

```
const names: string[] = [];  
names.push("Dylan");  
// names.push(3); // Erro: Argumento do tipo 'number'  
// não é atribuível para parâmetro do tipo 'string'.
```

- Readonly: impede que arrays sejam modificados;

```
const names: readonly string[] = ["Dylan"];  
names.push("Jack"); // Erro: 'push' não existe no tipo  
'readonly string[]'.
```

Tuplas

- ▶ É um array “tipado” com tipos predefinidos para cada índice;
- ▶ Permite que cada elemento no array tenha seu próprio tipo de valor;

```
// define uma tupla  
let tupla: [number, boolean, string];
```

```
// inicializa a tupla  
tupla = [10, true, 'Professor Arnaldo'];
```

Tuplas Nomeadas

- ▶ Permite prover contexto para os valores de cada índice:

```
const graph: [x: number, y: number] = [55.2, 41.3];
```

Objetos

- A sintaxe específica para definir objetos é:

```
const carro: { tipo: string, modelo: string, ano: number } = {  
  tipo: "Toyota",  
  modelo: "Corolla",  
  ano: 2009  
};
```

- Com propriedade opcional:

```
const carro: { marca: string, ano?: number } = { // sem erro  
  marca: "Toyota"  
};  
carro.ano = 2000;
```

Funções

- ▶ O tipo do valor de retorno precisa ser definido:

```
function getTime(): number {  
    return new Date().getTime();  
}
```

- ▶ Função sem retorno:

```
function printHello(): void {  
    console.log('Hello!');  
}
```

Funções

► Parâmetros opcionais:

```
function add(a: number, b: number, c?: number) {  
    return a + b + (c || 0);  
}
```

► Parâmetros padrões:

```
function pow(value: number, exponent: number = 10) {  
    return value ** exponent;  
}
```


União | (OR)

- Quando o valor pode ter mais de um tipo:

```
function printStatusCode(code: string | number) {  
    console.log(`My status code is ${code}.`)  
}  
printStatusCode(404);  
printStatusCode('404');
```


Casting (as)

- Sobrescreve o tipo de uma variável:

```
let x: unknown = 'hello';  
console.log((x as string).length);
```

ou

```
let x: unknown = 'hello';  
console.log((<string>x).length);
```

Classes

```
class Pessoa {  
    private nome: string;  
  
    public constructor(nome: string) {  
        this.nome = nome;  
    }  
  
    public getNome(): string {  
        return this.nome;  
    }  
}  
  
const pessoa = new Pessoa("Jane");  
console.log(pessoa.getNome());
```